

```
import time

class TSCHNode:
    def __init__(self, node_id, channels):
        self.node_id = node_id
        self.channels = channels
        self.num_channels = len(channels)
        self.asn = 0 # Absolute Slot Number

    def get_frequency(self, channel_offset):
        """Calcula o canal atual baseado no ASN e Offset"""
        index = (self.asn + channel_offset) % self.num_channels
        return self.channels[index]

    def run_slot(self, slot_type, channel_offset):
        """Simula a execução de um Time Slot"""
        freq = self.get_frequency(channel_offset)

        print(f"[ASN: {self.asn:03d}] Nó {self.node_id} no Canal {freq} MHz | Tipo: {slot_type}")

        # Lógica de transmissão/recepção
        if slot_type == "TX":
            print(f" >> Transmitindo dados...")
        elif slot_type == "RX":
            print(f" << Aguardando pacotes...")

        # Incrementa o ASN ao final de cada slot (Sincronização)
        self.asn += 1

# --- Configurações da Rede ---
IEEE_802_15_4_CHANNELS = [2405, 2410, 2415, 2420, 2425, 2430, 2435,
2440,
2445, 2450, 2455, 2460, 2465, 2470, 2475,
2480]

# Criando o nó (ex: um sensor de umidade)
sensor_node = TSCHNode(node_id="Sensor_01",
channels=IEEE_802_15_4_CHANNELS)

# Definição simples de um Slotframe (Ciclo de comunicação)
# 0 = TX, 1 = Dormir, 2 = Dormir
slotframe_config = {0: "TX", 1: "Sleep", 2: "Sleep"}
my_offset = 5 # Offset fixo para este link

print(f"Iniciando Simulação TSCH...\n")

try:
    for i in range(10): # Simula 10 slots de tempo
        current_slot = i % 3 # Tamanho do slotframe = 3

        action = slotframe_config.get(current_slot, "Sleep")

        if action != "Sleep":
            sensor_node.run_slot(action, my_offset)
        else:
            print(f"[ASN: {sensor_node.asn:03d}] Nó {sensor_node.node_id} em modo SLEEP (Economia de Energia)")
            sensor_node.asn += 1

        time.sleep(0.5) # Delay apenas para visualização no terminal

except KeyboardInterrupt:
    print("\nSimulação encerrada.")
```

(A)

```
import random

class TSCH_Blacklist_Node:
    def __init__(self, channels):
        self.asn = 0 # Absolute Slot Number
        self.all_channels = channels
        self.blacklist = set()
        self.channel_stats = {ch: {"success": 0, "total": 0} for ch
in channels}

    def get_current_channel(self, channel_offset):
        """
        Calcula o canal seguindo a fórmula TSCH:
        f = (ASN + channelOffset) mod n_channels
        Se o canal estiver na blacklist, ele busca o próximo
        disponível.
        """
        # 1. Tenta o canal original conforme o padrão TSCH
        base_idx = (self.asn + channel_offset) %
len(self.all_channels)
        target_ch = self.all_channels[base_idx]

        # 2. Se estiver na blacklist, faz o 're-mapping' para o
        próximo canal limpo
        if target_ch in self.blacklist:
            clean_channels = [c for c in self.all_channels if c not
in self.blacklist]
            if not clean_channels:
                return target_ch, "EMERGÊNCIA (Todos os canais
ruins)"

            # Re-mapeia para a lista limpa para garantir
            new_idx = (self.asn + channel_offset) %
len(clean_channels)
            return clean_channels[new_idx], "RE-MAPEADO (Blacklist
ativa)"

        return target_ch, "PADRÃO"

    def update_channel_health(self, channel, success):
        """Monitora o sucesso das transmissões para gerenciar a
        Blacklist"""
        stats = self.channel_stats[channel]
        stats["total"] += 1
        if success:
            stats["success"] += 1

        pdr = stats["success"] / stats["total"]

        # Lógica de entrada/saída da Blacklist (após 5 tentativas
        para ter base)
        if stats["total"] >= 5:
            if pdr < 0.6: # Taxa de sucesso menor que 60%
                self.blacklist.add(channel)
            elif pdr > 0.8: # Recuperação se o canal melhorar
                if channel in self.blacklist:
                    self.blacklist.remove(channel)

    def run_slot(self, offset):
        ch, mode = self.get_current_channel(offset)

        # Simula interferência (Canais 15 e 20 estão com ruído
        nesta simulação)
        interference = 0.9 if ch in [15, 20] else 0.1
        success = random.random() > interference

        self.update_channel_health(ch, success)

        status = "✓ OK" if success else "✗ FALHA"
        print(f"[ASN: {self.asn:03d}] Canal: {ch} | Modo:
{mode:<25} | Status: {status}")

        self.asn += 1

# --- Simulação ---
# Canais 11 a 26 (IEEE 802.15.4)
rf_channels = list(range(11, 27))
node = TSCH_Blacklist_Node(rf_channels)

print("Iniciando Rede TSCH com Blacklisting...\n")

for _ in range(40):
    node.run_slot(channel_offset=5)

print(f"\nLista Negra Final: {list(node.blacklist)}")
```

(B)